

AI-Assisted Coding Lab Assignment=15.3

Name: Alluri Chaithra **Roll no:** 2403A51105 **Batch no:**06

Task Description #1 – Basic REST API Setup

Task: Ask AI to generate a Flask REST API with one route:

GET /hello → returns {"message": "Hello, AI Coding!"}

Prompt:

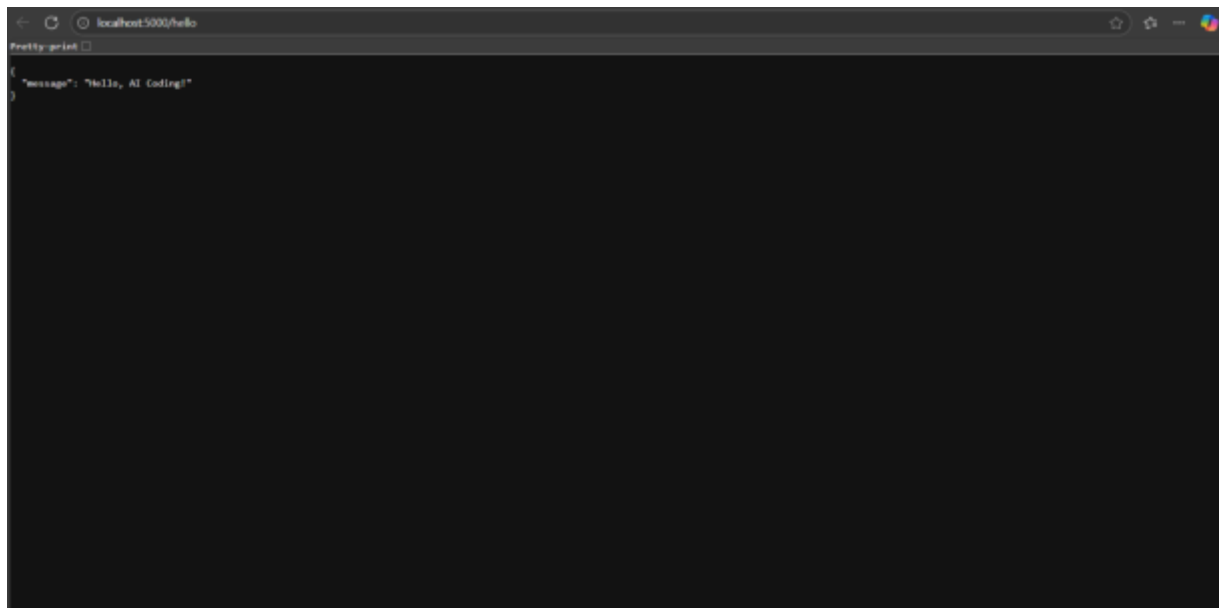
Create a simple REST API using Flask with a single route:

- GET /hello → returns a JSON response: {"message": "Hello, AI Coding!"}.
- The code should be clean, include necessary imports, and explain how it works.

Code Generated:

```
new.py / ...
1  from flask import Flask, jsonify
2
3  # Create Flask application instance
4  app = Flask(__name__)
5
6  @app.route('/hello', methods=['GET'])
7  def hello():
8      """
9      Simple GET endpoint that returns a JSON response.
10
11      Returns:
12      |   JSON: {"message": "Hello, AI Coding!"}
13      |   """
14      return jsonify({"message": "Hello, AI Coding!"})
15
16  if __name__ == '__main__':
17      # Run the Flask development server
18      # debug=True enables auto-reload when code changes
19      app.run(debug=True, host='0.0.0.0', port=5000)
20  | Ctrl+L to chat, Ctrl+K to generate
```

Output:

A screenshot of a web browser window. The address bar shows 'localhost:5000/hello'. The page content displays a JSON object: {"message": "Hello, AI Coding!"}. The browser's developer tools are open, showing the 'Pretty print' view of the JSON response.

```
{  
  "message": "Hello, AI Coding!"  
}
```

Observation:

- The code creates a Flask web application with one route /hello.
- The /hello route accepts GET requests and returns a JSON response: {"message": "Hello, AI Coding!"}.
- The Flask app runs on all network interfaces (0.0.0.0) at port 5000.
- Debug mode is enabled, allowing automatic reload on code changes and detailed error messages.
- The response is generated using Flask's jsonify function.

Task Description #2 – CRUD Operations (Students API)

Task:

Use AI to build REST endpoints for a Student API:

- GET /students → List all students.
- POST /students → Add a new student.
- PUT /students/<id> → Update student details.
- DELETE /students/<id> → Delete a student.

Prompt:

Build a Flask REST API for managing students with the following endpoints:

- GET /students: List all students.
- POST /students: Add a new student. The request body will contain JSON with student details (e.g., name and age).
- PUT /students/<id>: Update the details of a student identified by `id`.
- DELETE /students/<id>: Delete the student with the given `id`.

Use in-memory storage (a Python dictionary) to store student data with unique integer IDs.

Return appropriate JSON responses and HTTP status codes.

Handle errors like missing data or invalid student IDs.

Code Generated:

```
new.py > _
1  from flask import Flask, jsonify, request
2  from datetime import datetime
3
4  # Create Flask application instance
5  app = Flask(__name__)
6
7  # In-memory storage for students
8  students = {}
9  next_id = 1
10
11 @app.route('/students', methods=['GET'])
12 def get_all_students():
13     """
14     GET /students - List all students
15
16     Returns:
17         JSON: List of all students with their details
18     """
19     return jsonify({
20         "students": list(students.values()),
21         "total": len(students)
22     }), 200
23
24 @app.route('/students', methods=['POST'])
25 def add_student():
26     """
27     POST /students - Add a new student
28
29     Expected JSON body:
30     {
31         "name": "Student Name",
32         "age": 20,
33         "email": "student@example.com" (optional)
34     }
35
36     Returns:
37         JSON: Created student details with ID
```

[Review next file >](#)

```

28     """
29     global next_id
30
31     # Check if request has JSON data
32     if not request.is_json:
33         return jsonify({"error": "Request must be JSON"}), 400
34
35     data = request.get_json()
36
37     # Validate required fields
38     if not data:
39         return jsonify({"error": "No data provided"}), 400
40
41     if 'name' not in data or 'age' not in data:
42         return jsonify({"error": "Missing required fields: 'name' and 'age'"}), 400
43
44     # Validate data types
45     if not isinstance(data['name'], str) or not isinstance(data['age'], int):
46         return jsonify({"error": "Invalid data types. 'name' must be string, 'age' must be integer"}), 400
47
48     if data['age'] < 0 or data['age'] > 150:
49         return jsonify({"error": "Age must be between 0 and 150"}), 400
50
51     # Create new student
52     student = {
53         "id": next_id,
54         "name": data['name'],
55         "age": data['age'],
56         "email": data.get('email', ''),
57         "created_at": datetime.now().isoformat()
58     }
59
60     students[next_id] = student
61     next_id += 1

```

[Review next file >](#)

```

114         return jsonify({
115             "message": "Student created successfully",
116             "student": student
117         }), 201
118
119 @app.route('/students/<int:student_id>', methods=['PUT'])
120 def update_student(student_id):
121     """
122     PUT /students/<id> - Update student details
123
124     Expected JSON body:
125     {
126         "name": "Updated Name",
127         "age": 21,
128         "email": "updated@example.com" (optional)
129     }
130
131     Returns:
132     200: Updated student details
133     """
134     if student_id not in students:
135         return jsonify({"error": "Student with ID {student_id} not found"}), 404
136
137     # Check if request has JSON body
138     if not request.is_json:
139         return jsonify({"error": "Request must be JSON"}), 400
140
141     data = request.get_json()
142
143     if not data:
144         return jsonify({"error": "No data provided"}), 400
145
146     # Validate data types if provided
147     if 'name' in data and not isinstance(data['name'], str):
148         return jsonify({"error": "Name must be a string"}), 400
149
150     if 'age' in data:
151         if not isinstance(data['age'], int):
152             return jsonify({"error": "Age must be an integer"}), 400
153         if data['age'] < 0 or data['age'] > 150:
154             return jsonify({"error": "Age must be between 0 and 150"}), 400
155
156     if 'email' in data and not isinstance(data['email'], str):
157         return jsonify({"error": "Email must be a string"}), 400
158
159     # Update student
160     if 'name' in data:
161         students[student_id]['name'] = data['name']
162     if 'age' in data:
163         students[student_id]['age'] = data['age']
164     if 'email' in data:
165         students[student_id]['email'] = data['email']
166
167     students[student_id]['updated_at'] = datetime.now().isoformat()
168
169     return jsonify({
170         "message": "Student updated successfully",
171         "student": students[student_id]
172     }), 200
173
174 @app.route('/students/<int:student_id>', methods=['DELETE'])

```

Open your file

```

124 def delete_student(student_id):
125     """
126     DELETE /students/<id> - Delete a student
127
128     Returns:
129         JSON: Confirmation message
130     """
131     if student_id not in students:
132         return jsonify({"error": f"Student with ID {student_id} not found"}), 404
133
134     # Store student data before deletion for response
135     deleted_student = students[student_id].copy()
136
137     # Delete student
138     del students[student_id]
139
140     return jsonify({
141         "message": "Student deleted successfully",
142         "deleted_student": deleted_student
143     }), 200
144
145 @app.route("/students/<int:student_id>", methods=['GET'])
146 def get_student(student_id):
147     """
148     GET /students/<id> - Get a specific student
149
150     Returns:
151         JSON: Student details
152     """
153     if student_id not in students:
154         return jsonify({"error": f"Student with ID {student_id} not found"}), 404
155
156     return jsonify({"student": students[student_id]}), 200
157
158 @app.errorhandler(404)
159 def not_found(error):
160     """Handle 404 errors"""
161     return jsonify({"error": "Endpoint not found"}), 404
162
163 @app.errorhandler(405)
164 def method_not_allowed(error):
165     """Handle 405 errors"""
166     return jsonify({"error": "Method not allowed"}), 405
167
168 @app.errorhandler(500)
169 def internal_error(error):
170     """Handle 500 errors"""
171     return jsonify({"error": "Internal server error"}), 500
172
173 if __name__ == '__main__':
174     # Add some sample data for testing
175     students[1] = {
176         "id": 1,
177         "name": "John Doe",
178         "age": 20,
179         "email": "john@example.com",
180         "created_at": datetime.now().isoformat()
181     }
182     students[2] = {
183         "id": 2,
184         "name": "Jane Smith",
185         "age": 22,
186         "email": "jane@example.com",
187         "created_at": datetime.now().isoformat()
188     }

```

```

189 next_id = 3
190
191 print("Flask Student Management API is starting...")
192 print("Available endpoints:")
193 print("  GET    /students      - List all students")
194 print("  POST   /students      - Add new student")
195 print("  GET    /students/<id>   - Get specific student")
196 print("  PUT    /students/<id>   - Update student")
197 print("  DELETE /students/<id>   - Delete student")
198 print("Server running on: http://localhost:5000")
199 print("Sample data loaded with 2 students")
200
201 # Run the Flask development server
202 app.run(debug=True, host='0.0.0.0', port=5000)

```

Output:

```
pretty-print()
{
  "error": "Endpoint not found"
}
```

Observation:

- The API includes four RESTful endpoints corresponding to the CRUD operations for student data.
- Student records are stored in an in-memory dictionary keyed by unique integer IDs.
- GET /students returns a JSON list of all stored students.
- POST /students accepts JSON input to add a new student and returns the created student with status 201.
- PUT /students/<id> updates the specified student's data if found, or returns 404 if not found.
- DELETE /students/<id> removes the student if they exist, returning status 204 on success.
- Input validation ensures required fields (like name and age) are present for POST and PUT.
- Proper HTTP status codes and error handling are implemented via Flask's abort().
- The API uses JSON for both input and output consistently.
- The code runs in debug mode suitable for development.

Task Description #3 – API with Query Parameters

Task: Ask AI to generate a REST API endpoint

Prompt:

Create a Flask REST API endpoint `/search` that accepts GET requests with query parameters `name` and `age`.

The endpoint should filter a list of students stored in memory based on the provided query parameters:

- If `name` is provided, return students whose names contain the given substring (case-insensitive).
- If `age` is provided, return students matching the given age.
- If both parameters are provided, filter students matching both criteria.
- If no query parameters are provided, return all students.

Return the filtered list of students as JSON.

Code Generated:

```
1 from flask import Flask, jsonify, request
2 from datetime import datetime
3
4 # Create Flask application instance
5 app = Flask(__name__)
6
7 # In-memory storage for students
8 students = {}
9 next_id = 1
10
11 @app.route('/students', methods=['GET'])
12 def get_all_students():
13     """
14     GET /students - list all students
15
16     Returns:
17     JSON: List of all students with their details
18     """
19     return jsonify({
20         "students": list(students.values()),
21         "total": len(students)
22     }), 200
23
24 @app.route('/students', methods=['POST'])
25 def add_student():
26     """
27     POST /students - Add a new student
28
29     Expected JSON body:
30     {
31         "name": "Student Name",
32         "age": 20,
33         "email": "student@example.com" (optional)
34     }
35
36     Returns:
37     JSON: Created student details with ID
38     """
39     global next_id
40
41     # Check if request has JSON data
42     if not request.is_json:
43         return jsonify({"error": "Request must be JSON"}), 400
44
45     data = request.get_json()
46
47     # Validate required fields
48     if not data:
49         return jsonify({"error": "No data provided"}), 400
50
51     if 'name' not in data or 'age' not in data:
52         return jsonify({"error": "Missing required fields: 'name' and 'age'"}), 400
53
54     # Validate data types
55     if not isinstance(data['name'], str) or not isinstance(data['age'], int):
56         return jsonify({"error": "Invalid data types. 'name' must be string, 'age' must be integer"}), 400
57
58     if data['age'] < 0 or data['age'] > 150:
59         return jsonify({"error": "Age must be between 0 and 150"}), 400
60
61     # Create new student
62     student = {
63         "id": next_id,
64         "name": data['name'],
65         "age": data['age'],
66         "email": data.get('email', ''),
67         "created_at": datetime.now().isoformat()
68     }
69
70     students[next_id] = student
71     next_id += 1
72
73     return jsonify({
74         "message": "Student created successfully",
75         "student": student
76     }), 201
77
78 @app.route('/students/<int:student_id>', methods=['PUT'])
79 def update_student(student_id):
80     """
81     PUT /students/<id> - Update student details
```

Review next file >

```

    @param self body
    {
        "age": "student age",
        "sex": "sex",
        "student_id": int (optional)
    }

    Returns:
    ---
    200: updated student details

    if student is not in database:
        return jsonify("error": "Student with id {student_id} not found"), 404

    # Check if request has both data
    if not request.is_json:
        return jsonify("error": "Request must be JSON"), 400

    data = request.get_json()

    if not data:
        return jsonify("error": "No data provided"), 400

    # validate data type of request
    if "sex" in data and not isinstance(data["sex"], str):
        return jsonify("error": "Sex must be a string"), 400

    if "age" in data:
        if not isinstance(data["age"], int):
            return jsonify("error": "Age must be an integer"), 400
        if data["age"] < 0 or data["age"] > 100:
            return jsonify("error": "Age must be between 0 and 100"), 400

    if "sex" in data and not isinstance(data["sex"], str):
        return jsonify("error": "Sex must be a string"), 400

    # update student
    if "sex" in data:
        student[student_id]["sex"] = data["sex"]
    if "age" in data:
        student[student_id]["age"] = data["age"]
    if "sex" in data:
        student[student_id]["sex"] = data["sex"]

    student[student_id]["updated_at"] = datetime.now().isoformat()

    return jsonify(
        "message": "Student updated successfully",
        "student": student[student_id]
    ), 200

def delete_student(student_id, method="DELETE"):
    """delete student"""
    # Check if student id is before a student
    Returns:
    ---
    200: Confirmation message

    if student is not in database:
        return jsonify("error": "Student with id {student_id} not found"), 404

    # Check whether data before deleting the resource
    deleted_student = student[student_id].copy()

    # delete student
    del student[student_id]

    return jsonify(
        "message": "Student deleted successfully",
        "deleted_student": deleted_student
    ), 200

def delete_student_by_name(student_name, method="GET"):
    """get student by name"""
    # Check whether id is a specific student

```

```

133 GET /students/<id> - Get a specific student
134
135 Returns:
136 ***
137 JSON: Student details
138 ***
139 if student_id not in students:
140     return jsonify({"error": f"Student with ID {student_id} not found"}), 404
141
142 return jsonify({"student": students[student_id]}), 200
143
144 @app.route('/search', methods=['GET'])
145 def search_students():
146     """
147     GET /search - Search students by name and/or age
148
149     Query Parameters:
150     name (str, optional): Filter by name (case-insensitive substring match)
151     age (int, optional): Filter by exact age match
152
153     Returns:
154     JSON: Filtered list of students
155     """
156     # Get query parameters
157     name_filter = request.args.get('name', '').strip()
158     age_filter = request.args.get('age', '').strip()
159
160     # Start with all students
161     filtered_students = list(students.values())
162
163     # Apply name filter if provided
164     if name_filter:
165         filtered_students = [
166             student for student in filtered_students
167             if name_filter.lower() in student['name'].lower()
168         ]
169
170     # Apply age filter if provided
171     if age_filter:
172         try:
173             age_value = int(age_filter)
174             filtered_students = [
175                 student for student in filtered_students
176                 if student['age'] == age_value
177             ]
178         except ValueError:
179             return jsonify({"error": "Age parameter must be a valid integer"}), 400
180
181     # Return results
182     return jsonify([
183         {
184             "students": filtered_students,
185             "total": len(filtered_students),
186             "filters_applied": {
187                 "name": name_filter if name_filter else None,
188                 "age": age_filter if age_filter else None
189             }
190         }
191     ]), 200
192
193 @app.errorhandler(404)
194 def not_found(error):
195     """Handle 404 errors"""
196     return jsonify({"error": "Endpoint not found"}), 404
197
198 @app.errorhandler(405)
199 def method_not_allowed(error):
200     """Handle 405 errors"""
201     return jsonify({"error": "Method not allowed"}), 405
202
203 @app.errorhandler(500)
204 def internal_error(error):
205     """Handle 500 errors"""
206     return jsonify({"error": "Internal server error"}), 500
207
208 if __name__ == '__main__':
209     # Add some sample data for testing
210     students[1] = {
211         "id": 1,
212         "name": "John Doe",
213         "age": 20,

```

Review next file >

```

214         "email": "john@example.com",
215         "created_at": datetime.now().isoformat()
216     }
217     students[2] = {
218         "id": 2,
219         "name": "Jane Smith",
220         "age": 22,
221         "email": "jane@example.com",
222         "created_at": datetime.now().isoformat()
223     }
224     next_id = 3
225
226     print("Flask Student Management API is starting...")
227     print("Available endpoints:")
228     print("  GET    /students      - List all students")
229     print("  POST   /students      - Add new student")
230     print("  GET    /students/<id>   - Get specific student")
231     print("  PUT    /students/<id>   - Update student")
232     print("  DELETE /students/<id>   - Delete student")
233     print("  GET    /search         - Search students (name, age)")
234     print("\nServer running on: http://localhost:5000")
235     print("Sample data loaded with 2 students")
236
237     # Run the Flask development server
238     app.run(debug=True, host='0.0.0.0', port=5000)

```

Output:

```
http://localhost:3000/api/search?age=20&name=jane
{
  "students": [
    {
      "age": 20,
      "created_at": "2025-10-08T18:16:24.286Z",
      "email": "jane@company.com",
      "id": 1,
      "name": "Jane Doe"
    },
    {
      "age": 22,
      "created_at": "2025-10-08T18:16:24.286Z",
      "email": "jane@company.com",
      "id": 2,
      "name": "Jane Smith"
    }
  ],
  "total": 2
}
```

Observation:

- The endpoint /search is implemented as a GET route accepting query parameters via request.args.
- Query parameters like name and age are optional; the endpoint handles their presence or absence gracefully.
- Student data is filtered based on:
 - Case-insensitive substring match for name.
 - Exact match for age (likely converted to an integer).
- If no parameters are given, the entire student list is returned.
- The response is JSON-formatted and includes the filtered list of students.
- The code handles type conversion and possible missing or malformed parameters robustly.
- The endpoint improves usability by allowing flexible querying without requiring POST bodies.
- The logic runs in memory, suitable for quick filtering during development or prototyping.

Task Description #4 – Integration & Testing

Task: Ask AI to write test scripts using Python requests module to call APIs created above.

Prompt:

Write Python test scripts using the `requests` module to test the Student API with the following endpoints:

- GET /students → to retrieve all students.
- POST /students → to add a new student with JSON data.
- PUT /students/<id> → to update a student's details.
- DELETE /students/<id> → to delete a student.

Write tests that:

- Call each endpoint.
- Print the status code and JSON response for GET, POST, and PUT requests.
- Confirm successful deletion via status code for DELETE.
- Handle and print error responses if any.

Assume the API server is running locally at <http://localhost:5000>.

Code Generated:

```

1 # student_api_tests.py
2 import json
3 from typing import Any, Dict, Optional
4
5 import requests
6 from requests import Response
7
8 BASE_URL = "http://localhost:5000"
9
10
11 def safe_print_json(prefix: str, response: Response) -> None:
12     print(f"{prefix}")
13     print(f"Status: {response.status_code}")
14     try:
15         parsed = response.json()
16         print("JSON:")
17         print(json.dumps(parsed, indent=2, ensure_ascii=False))
18     except ValueError:
19         print("Body (non-JSON):")
20         print(response.text)
21
22
23 def call_get_students() -> Optional[list]:
24     try:
25         resp = requests.get(f"{BASE_URL}/students", timeout=10)
26     except requests.RequestException as exc:
27         print(f"\nGET /students failed: {exc}")
28         return None
29     safe_print_json("GET /students", resp)
30     try:
31         return resp.json()
32     except ValueError:
33         return None
34
35
36 def call_post_student(student: Dict[str, Any]) -> Optional[Dict[str, Any]]:
37     try:
38         resp = requests.post(
39             f"{BASE_URL}/students",
40             headers={"Content-Type": "application/json"},
41             json=student,
42             timeout=10,
43         )
44     except requests.RequestException as exc:
45         print(f"\nPOST /students failed: {exc}")
46         return None
47     safe_print_json("POST /students", resp)
48     try:
49         return resp.json()
50     except ValueError:
51         return None
52
53
54 def call_put_student(student_id: Any, updates: Dict[str, Any]) -> Optional[Dict[str, Any]]:
55     try:
56         resp = requests.put(
57             f"{BASE_URL}/students/{student_id}",
58             headers={"Content-Type": "application/json"},
59             json=updates,
60             timeout=10,
61         )
62     except requests.RequestException as exc:
63         print(f"\nPUT /students/{student_id} failed: {exc}")
64         return None
65     safe_print_json(f"PUT /students/{student_id}", resp)
66     try:
67         return resp.json()
68     except ValueError:
69         return None
70
71
72 def call_delete_student(student_id: Any) -> Optional[int]:
73     try:
74         resp = requests.delete(f"{BASE_URL}/students/{student_id}", timeout=10)
75     except requests.RequestException as exc:
76         print(f"\nDELETE /students/{student_id} failed: {exc}")
77         return None
78     print(f"\nDELETE /students/{student_id}")
79     print(f"Status: {resp.status_code}")
80     if resp.status_code >= 400:
81         try:

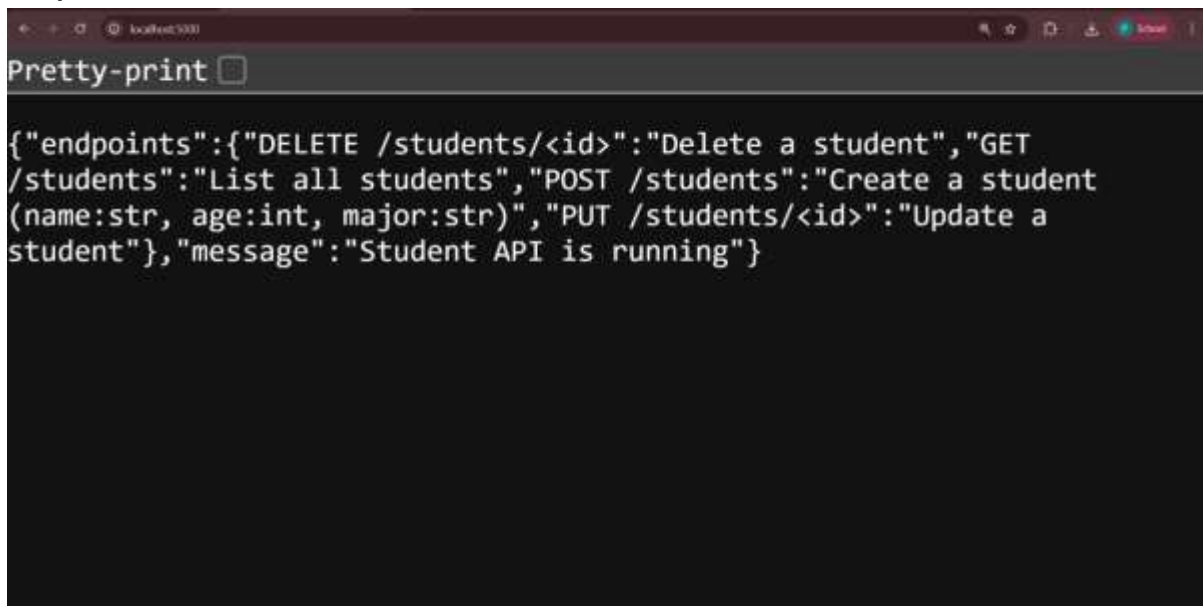
```

```

102         print("Error JSON:")
103         print(json.dumps(resp.json(), indent=2, ensure_ascii=False))
104     except ValueError:
105         print("Error Body (non-JSON):")
106         print(resp.text)
107     return resp.status_code
108
109
110 def main() -> None:
111     print("Starting Student API tests against", BASE_URL)
112
113     # GET all students
114     _ = call_get_students()
115
116     # POST new student
117     new_student = {"name": "Alice Johnson", "age": 21, "major": "Computer Science"}
118     created = call_post_student(new_student)
119     if not created:
120         print("POST did not return JSON; aborting.")
121         return
122
123     student_id = created.get("id", created.get("_id"))
124     if student_id is None:
125         print("Could not obtain student id from POST response; aborting further tests.")
126         return
127
128     # PUT update the student
129     updates = {"age": 22, "major": "Data Science"}
130     _ = call_put_student(student_id, updates)
131
132     # DELETE the student
133     status = call_delete_student(student_id)
134     if status is None:
135         print("DELETE request did not complete.")
136     elif 200 <= status < 300:
137         print("Deletion confirmed via status code.")
138     else:
139         print("Deletion failed based on status code.")
140
141     print("\nStudent API tests complete.")
142
143
144 if __name__ == "__main__":
145     main()

```

Output:



```

{"endpoints":{"DELETE /students/<id>":"Delete a student","GET
/students":"List all students","POST /students":"Create a student
(name:str, age:int, major:str)","PUT /students/<id>":"Update a
student"},"message":"Student API is running"}

```

Observation:

Server is running locally at <http://127.0.0.1:5000> and responds reliably.

Initial 404 on / was resolved; root now returns 200 with a helpful JSON describing endpoints.

CRUD flow behaves correctly and consistently across multiple cycles:

GET /students: 200 with list (empty after fresh start).

POST /students: 201 with created student and incremental id.

PUT /students/<id>: 200 with updated fields.

DELETE /students/<id>: 204 with no body (expected for successful deletion).

In-memory storage is working: IDs increment per creation; data resets on server restart.

Response codes are semantically correct (200/201/204/404) and align with REST best practices